

Cost Optimization at Scale

Prompt Caching · Semantic Caching · Model Routing · Batching Economics · Compression
· Cost Attribution

WEEK 4 · DAY 24

Wednesday, July 29 2026

Day 24 of 84

Three Observations From Day 21's Measured Token Profile

RetrievalAgent (8B): in=1,850 out=180
GenerationAgent (70B): in=2,400 out=320
ValidationAgent (8B): in=1,200 out=140
TOTAL: in=5,450 out=640

- 1. INPUT IS 8.5x OUTPUT. You pay overwhelmingly to send context IN.
Every instinct that targets output length aims at 10% of the problem.
- 2. THE SAME SCHEMA CONTEXT IS SENT ON EVERY REQUEST. A tenant's schema does not change between queries, yet we re-transmit and re-process the identical ~2,000 tokens thousands of times a day.
- 3. EVERY QUERY PAYS FOR THE 70B MODEL. "Count all users" and "revenue by cohort with a 3-table anti-join" cost exactly the same.

TODAY CLOSES ALL THREE.

Sec	Topic	Coverage
Sec 1	Cost Anatomy	CostBreakdown, stable_fraction, optimization order
Sec 2	Prompt Caching	Prefix ordering, CacheEconomics, the 5 cache killers
Sec 3	Semantic Caching	Threshold calibration, isolation, verified-only writes
Sec 4	Model Routing	ComplexityRouter, EscalatingGenerator, per-tier monitoring
Sec 5	Batching + Compression	Throughput ladder, compression-vs-caching conflict
Sec 6	Cost Attribution	CostAttributor, BudgetEnforcer, the 4 questions
Sec 7	Combined Result	-57.2% cost for -1.7pp accuracy; the accuracy budget
Sec 8	FAANG Practice	Google/Microsoft/Amazon/Meta/Netflix cost practice
Sec 9	Architect's Lens	Optimization order, self-hosted scoreboard, fail safe

Sec 10	Exercises + Summary	Cache killer, threshold, routing, phantom queries
--------	---------------------	---

Section 1 — Cost Anatomy: Know Where the Money Is

Before optimizing anything, decompose the cost. Optimizing the wrong component is the most common waste of engineering time in this area.

[illegible]

[illegible]

[illegible]

Section 4 — Model Routing by Query Complexity

Day 20 mixed model sizes by AGENT ROLE. Today we mix by QUERY DIFFICULTY within the generation role itself. Day 21's difficulty breakdown is the evidence: easy queries score 95.9% — does an 8B model also get those right?

```
class ComplexityRouter:
    """
    Routes using SIGNALS AVAILABLE BEFORE GENERATION. This constraint shapes
    the whole design: you must decide the tier WITHOUT having generated the
    SQL, so you cannot use the SQL.

    Signals you DO have after retrieval (Day 17/20):
    - number of tables retrieved
    - whether joins are implied by the retrieved schema
    - question length and clause markers
    - whether a similar question exists in long-term memory (Day 19)
    """
    JOIN_MARKERS = ("per ", "by ", "for each", "grouped", "across", "compared",
                    "versus", " vs ", "along with")
    HARD_MARKERS = ("except", "not in", "neither", "excluding", "without any",
                   "more than average", "top ", "rank", "percentile",
                   "year over year", "cohort", "running total", "cumulative")

    def score(self, question, retrieved_tables, implied_joins, has_memory_hit):
        q = question.lower()
        signals = {"n_tables": len(retrieved_tables), "n_joins": len(implied_joins),
                  "hard_markers": sum(1 for m in self.HARD_MARKERS if m in q),
                  "join_markers": sum(1 for m in self.JOIN_MARKERS if m in q),
                  "q_words": len(q.split()), "memory_hit": has_memory_hit}
        complexity = (max(0, signals["n_tables"]-1)*2.0 +
                     signals["n_joins"]*1.5 +
                     signals["hard_markers"]*3.0 +
                     signals["join_markers"]*0.5 +
                     (1.0 if signals["q_words"] > 25 else 0.0) -
                     (1.5 if signals["memory_hit"] else 0.0))
        # A memory hit means this exact shape was answered correctly before —
        # the small model has a TEMPLATE to follow, which is precisely when a
        # small model performs closest to a large one.
        tier = Tier.SMALL if complexity <= 3.0 else Tier.LARGE
        if signals["n_tables"] > self.cfg.max_small_tables:
            tier = Tier.LARGE # hard override: many tables ALWAYS go LARGE
        return {"tier": tier, "complexity": round(complexity, 2), "signals": signals}

class EscalatingGenerator:
    """Try SMALL. If the result fails validation, retry on LARGE.
    This is the safety net that makes routing acceptable: a misrouted query
    costs extra LATENCY, not a WRONG ANSWER."""
    async def generate(self, question, schema_context, routing) -> dict:
        if routing["tier"] == Tier.SMALL:
            result = await self.small.generate(question, schema_context)
            check = await self.validator.check(result["sql"], schema_context)
            if check["is_valid"] and check["is_faithful"]:
                model_tier_used.labels(tier="small", escalated="false").inc()
                return {**result, "tier": "small", "escalated": False}
            # Escalate. We pay for BOTH calls on this query.
            model_tier_used.labels(tier="large", escalated="true").inc()
            return {**(await self.large.generate(question, schema_context)),
```


[illegible]

Section 5 — Batching Economics and Context Compression

On self-hosted GPUs your cost is TIME, not tokens (Day 21: \$5.67/hr divided by queries served). Cost per query is entirely determined by throughput, and throughput is determined by batch efficiency.

Serving Mode	Queries/hr	Cost/query	Note
Sequential, no batching	210	\$0.027000	4.3x worse than baseline
Static batching (8)	480	\$0.011813	Padding waste
Continuous batching	900	\$0.006300	Day 21's measured number
+ prefix caching	1,240	\$0.004573	Today's improvement

```
# On self-hosted, prompt caching shows up as THROUGHPUT, not as a
# discounted invoice line – you skip prefill compute, so the GPU serves
# more requests per hour. Same optimization, DIFFERENT ACCOUNTING.
# If you measure only tokens, you will conclude that caching did nothing.
```

```

class SchemaContextCompressor:
    """
    Day 17 introduced contextual compression for retrieval. Here it reduces
    prompt COST.

    IMPORTANT: COMPRESSION AND CACHING FIGHT EACH OTHER.
    Question-specific compression makes the schema block vary per request,
    which DESTROYS the prefix cache. Choose deliberately:

        SMALL schema (< 3k tokens) -> CACHE IT WHOLE. No compression.
        HUGE schema  (> 20k tokens) -> COMPRESS. The cache was never going
                                         to hold it anyway.

    In between -> cache the stable core tables, compress the long tail.
    """
    def compress_ddl(self, ddl: str, relevant_columns: set[str]) -> str:
        """
        Lossy but safe reductions, in order of savings:
            1. Drop comments and formatting whitespace           ~8%
            2. Drop columns not in relevant_columns              ~35% on wide tables
            3. Drop constraint clauses not needed for joins      ~12%
            4. Abbreviate types (CHARACTER VARYING->VARCHAR)     ~4%
        NEVER drop: primary keys, foreign keys, or any column named in
        relevant_columns. Dropping a JOIN KEY produces a confidently wrong
        query, which is far more expensive than the tokens saved.
        """

# ■■■ OUTPUT-SIDE LEVERS (smallest; listed for completeness) ■■■■■■■■■■■■
# max_tokens ceiling – prevents runaway generation; a SAFETY control
#                      more than a cost control
# stop sequences     – cut generation at the SQL terminator
# CoT length limits  – "reason in at most 5 short steps"
#
# DO NOT REMOVE CoT TO SAVE TOKENS. Day 21's ablation: -10.5pp accuracy
# for ~120 output tokens. That trade is catastrophic and teams make it.

```


Section 7 — The Combined Result

Optimization	Cost/query	Delta	Accuracy Cost	Risk
Baseline (Day 21 measured)	\$0.006300	—	—	—
+ Prompt caching	\$0.004573	-27.4%	0.0pp	None
+ Model routing (12% escal.)	\$0.003570	-21.9%	-1.4pp	Low
+ Semantic caching (18% hit)	\$0.002927	-18.0%	-0.3pp	Medium
+ Context compression	\$0.002694	-8.0%	0.0pp	Low
FINAL	\$0.002694	-57.2%	-1.7pp	—

SLO CHECK: 84.5% - 1.7pp = 82.8% > 82.0% target PASS

READ THIS LIKE AN ARCHITECT:

The accuracy budget was 2.5pp (84.5% headroom above the 82% SLO). We spent 1.7pp of it and bought a 57% cost reduction. That is a GOOD TRADE — but it IS a trade, and it is now SPENT. The next optimization has only 0.8pp of headroom left before the system violates its SLO.

This is the discipline: cost optimization DRAWS DOWN an accuracy budget. Track the remaining balance explicitly, or you will optimize your way through the SLO one "small" change at a time.

ORDER MATTERS TOO: prompt caching was FIRST because it costs ZERO accuracy. Always exhaust the FREE optimizations before spending accuracy on the paid ones.

Section 8 — FAANG Cost Practice

Google — Context Caching in the Gemini API

Google exposes context caching as an explicit, first-class object: you create a cached content resource containing a large stable context, receive a handle, and reference that handle across subsequent requests, with cached input tokens billed at a reduced rate plus a storage charge for the cache's lifetime. The design difference worth noting is that Google's cache is EXPLICITLY CREATED AND NAMED rather than inferred from a prefix match, making cache lifetime something you manage deliberately rather than something that happens to you. For Text-to-SQL this maps cleanly onto the tenant schema — one cached context per tenant, refreshed when the schema fingerprint changes, which is exactly the invalidation signal Day 22's corpus drift detector already computes.

Microsoft — Provisioned Throughput Versus Pay-As-You-Go

Azure OpenAI offers both consumption billing and Provisioned Throughput Units (PTUs), where you reserve dedicated capacity for a fixed cost regardless of how much you use. This is the managed-service version of Day 21's self-hosted-versus-API breakeven: below a sustained utilization threshold consumption wins; above it, reserved capacity wins and the incremental request is effectively free. The architectural consequence is that once you are on reserved capacity your optimization target INVERTS — you stop trying to reduce tokens and start trying to MAXIMIZE UTILIZATION of capacity you already paid for, the same accounting shift that makes batching the dominant lever on self-hosted GPUs.

Amazon — Bedrock Prompt Caching and Batch Inference

Bedrock supports prompt caching with explicit cache checkpoints marking reusable prefixes, and separately offers BATCH INFERENCE for workloads that tolerate delayed completion at a significantly reduced per-token price. The batch option is the lever most teams overlook because it does not apply to interactive traffic — but a large share of real LLM spend is NOT interactive. Nightly report generation, backfilling embeddings, evaluation runs, and bulk classification are all latency-insensitive. Day 22's continuous canary suite is a perfect batch candidate: it runs hourly, nobody is waiting on it, and it is pure recurring cost.

Meta — Small Models Where Small Models Suffice

Meta's release strategy for Llama has consistently shipped a RANGE of parameter sizes in each generation rather than a single flagship, with the stated rationale that smaller models are the right deployment choice for a large class of tasks, particularly when fine-tuned or given good context. This is the empirical foundation under Section 4's router: the question is never 'which model is best' but 'which is the SMALLEST model that clears the accuracy bar for THIS query.' Meta's smaller Llama variants, combined with the distillation approach in recent releases, exist precisely so production systems can serve most traffic without paying flagship inference cost.

Netflix — Cost Attribution as an Engineering Discipline

Netflix's engineering writing on cloud efficiency has long emphasized attributing infrastructure cost back to the owning team and service, on the argument that cost INVISIBLE to the team that generates it is cost NOBODY optimizes. Applied to LLM systems, the equivalent is per-feature and per-tenant token attribution — the CostAttributor in Section 6. The practical payoff is specific and repeatable: the first time a team runs that report, the single largest line item is almost always something nobody knew was running at that volume, and turning it off or fixing it delivers a bigger saving than any of the clever optimizations in this lesson.

Section 9 — Architect's Lens

THE OPTIMIZATION ORDER – and why deviating from it wastes effort

FREE (zero accuracy cost) – DO ALL OF THESE FIRST

1. Prompt prefix ordering (one afternoon, ~27%)
2. Deterministic serialization (one hour, UNLOCKS #1)
3. Remove timestamps from prompts (five minutes, UNLOCKS #1)
4. Continuous batching (already done – Day 11)
5. Batch API for non-interactive (canary suite, backfills)

CHEAP (small accuracy cost, bounded risk)

6. Model routing with escalation (-1.4pp, ~22%)
7. Context compression, wide tenants (-0.0pp if join keys kept)

EXPENSIVE (real risk, needs a calibration harness)

8. Semantic caching (-0.3pp, ~18%, needs 200 pairs)

NEVER

9. Removing CoT to save output tokens (-10.5pp for ~2% savings)
10. Lowering the semantic threshold to chase hit rate
11. Sharing any cache across tenants

Most teams start at #6 or #8 because they are INTERESTING.

#1-#3 take one day, cost NO accuracy, and deliver a THIRD of the total.

THE ACCURACY BUDGET – the concept that keeps this honest

current_accuracy - slo_floor = your budget

84.5% - 82.0% = 2.5pp available

Every cost optimization is a WITHDRAWAL from this account.

Track the balance in the SAME dashboard as the cost number, or the two get optimized independently and the SLO loses.

WHAT CHANGES WHEN YOU ARE SELF-HOSTED

Managed API: cost = f(tokens). Optimize TOKENS.

Self-hosted: cost = f(GPU-hours). Optimize THROUGHPUT.

Same techniques, different scoreboard. Prompt caching on self-hosted does not produce a discounted invoice line – it produces 1,240 q/hr instead of 900. If you measure only tokens, you will conclude caching did nothing.

THE QUESTION THAT FINDS THE REAL WASTE

Not "how do we make each query cheaper?" but

"WHICH QUERIES SHOULD NOT BE RUNNING AT ALL?"

In practice: a retry storm, an agent loop without a step cap (Day 19's MAX_STEPS exists for this), a dashboard polling every 30 seconds for data that changes daily, or an eval suite left at 5-minute intervals.

Check for these BEFORE optimizing token counts. They are usually larger.

COST OPTIMIZATION MUST FAIL SAFE

Every mechanism here needs an automatic OFF SWITCH:

small-tier accuracy < floor -> route 100% to LARGE, alert

semantic FP rate > 1% -> disable cache, alert

compression drops a join key -> hard error, NEVER silent

A cost optimization that degrades quality silently is indistinguishable from Day 22's Class 2 failure – because that is exactly what it is.

Section 10 — Exercises and Summary

Exercise 1: Find Your Cache Killer

```
# Log the EXACT prompt string sent for 100 consecutive requests.
# Compute the length of the COMMON PREFIX across all of them.
# common_prefix_len / avg_prompt_len = your theoretical cache ceiling.
# If it is under 50%, find what varies early – it is almost always a
# timestamp, a UUID, or non-deterministic dict/set ordering.
# Fix it and re-measure BEFORE touching anything else.
```

Exercise 2: Calibrate the Semantic Threshold

```
# Build 200 (question_a, question_b, should_share_answer) pairs.
# Include at least 40 ADVERSARIAL near-misses – the "2024 vs 2025",
# "top 10 vs top 100", "region vs country" family.
# Run calibrate_threshold(). Plot false-positive rate vs hit rate.
# Pick the HIGHEST threshold with acceptable hit rate.
# Report the FP rate you are accepting as an EXPLICIT decision.
```

Exercise 3: Measure Routing Honestly

```
# Run your Day 21 eval set through the 8B model ALONE.
# Record accuracy BY DIFFICULTY, not in aggregate.
# For each band compute: accuracy_delta vs cost_delta.
# Route only the bands where the trade is favorable.
# Expected: easy is clearly worth routing; extra clearly is not;
# medium is the judgment call and depends on your SLO headroom.
```

Exercise 4: Find the Query That Should Not Exist

```
# Group one week of cost by (feature, tenant) and sort descending.
# Investigate the top 3 lines. For each ask: "is this traffic NECESSARY?"
# Look specifically for: polling loops, uncapped retries, agent runs
# hitting MAX_STEPS, eval suites on too-frequent schedules.
# Report the saving from ELIMINATING waste vs OPTIMIZING tokens.
# In most systems the first number is larger, and it costs no accuracy.
```

Summary

Concept	Key Point
The core observation	Input is 8.5x output; 73% of input is identical every request
CostBreakdown	Decompose before optimizing; stable_fraction() is your caching ceiling
Optimization order	Caching → routing → semantic cache → compression → batching → output
Prompt caching	Works on PREFIXES; order most-stable to least-stable
The ordering mistake	Question first = 0% hit rate; one line costs the whole win
CacheEconomics	Breakeven hit rate ~8% — essentially any real workload profits
Cache killers	Timestamps, early few-shot, UUIDs, unsorted serialization, TTL
Semantic caching	Skips inference entirely; highest savings AND highest risk

Threshold 0.97	0.85 and 0.92 produce confidently wrong answers on near-misses
calibrate_threshold	200 pairs with adversarial near-misses; run BEFORE enabling
Cache verified only	Caching a wrong answer makes a one-time error permanent
Namespace isolation	Never share a semantic cache across tenants — it is a data leak
ComplexityRouter	Pre-generation signals only: tables, joins, markers, memory hit
EscalatingGenerator	Small fails validation → retry large; misroute costs latency
Escalation economics	Profitable to 70% escalation; latency budget binds before cost
Per-tier accuracy	Aggregate hides small-tier decay; canary must run per tier
BatchEconomics	Self-hosted cost is time; caching shows up as throughput
Throughput ladder	210 → 480 → 900 → 1,240 q/hr (sequential → +prefix caching)
Compression vs caching	They fight; compress huge schemas, cache small ones
Never drop join keys	A wrong query costs far more than the tokens saved
Never remove CoT	-10.5pp accuracy for ~2% savings (Day 21 ablation)
CostAttributor	Unattributed cost is unmanageable cost
BudgetEnforcer	Degrade before deny; alerting is not enforcement
Combined result	-57.2% cost for -1.7pp accuracy; 82.8% still clears the 82% SLO
Accuracy budget	current - slo_floor; every optimization is a withdrawal
Fail safe	Every mechanism needs an auto-off switch, or it is a Class 2 failure
The real waste	"Which queries should not run at all?" beats cheaper queries

NEXT: Day 25 — Security and Compliance for LLM Systems

Prompt injection defense, data residency, PII handling in prompts and logs, audit trails for regulated environments, and the threat model for a system that executes machine-generated code against production data.